

Environment & Autoloader

Van hardcoded spaghetti naar een nette projectstructuur

Leerjaar	Jaar 1 · Periode 2
Doel	PSR-4 autoloading, namespaces, .env en het singleton-patroon stap voor stap opbouwen
Werkwijze	Individueel · gebruik je eigen IDE en XAMPP
Inleveren	PHP-map gezipd via de gebruikelijke manier

Wat ga je bouwen?

Je begint met de meest simpele versie die werkt: een paar regels PHP die praten met de database. Daarna verbeter je die code stap voor stap. Elke stap lost een concreet probleem op.

Aan het einde heb je:

- omgevingsvariabelen in een `.env`-bestand
- automatische class-loading via Composer (PSR-4)
- een `Database`-klasse met het singleton-patroon
- een `UserRepository` die alle SQL afhandelt

Vorbereiding: database importeren

Importeer `user_repo_opdracht.sql` via phpMyAdmin: **Importeren** → **bestand kiezen** → **Uitvoeren**.

Stap 1: Het werkt, maar het is lelijk

Maak een nieuwe map aan met de naam `Environment` in je XAMPP `htdocs`-map. Maak daarin `index.php` aan:

```
<?php

$pdo = new PDO('mysql:host=localhost;dbname=user_repo_opdracht', 'root', '');
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute(['peter']);
$user = $stmt->fetch();

var_dump($user);
```

Open de browser en ga naar `localhost/Environment/index.php`.

Het werkt. Maar het wachtwoord staat hardcoded in je code. Als je dit op GitHub zet, ziet iedereen je databasewachtwoord.

Stap 2: Wachtwoorden uit de code

Probleem: credentials staan letterlijk in je code. **Oplossing:** sla ze op in een `.env`-bestand en laad ze in via `phpdotenv`.

Composer initialiseren

Open een terminal in de `Environment`-map en voer uit:

```
composer init
```

Vul in bij **Package name:** `php/app`. De rest: Enter om over te slaan. Composer genereert een `composer.json` met daarin al een `autoload PSR-4` sectie. Pas de namespace daarin aan naar `App\`:

```
{
    "name": "php/app",
    "autoload": {
        "psr-4": {
            "App\\": "src/"
        }
    },
    "require": {}
}
```

phpdotenv installeren

```
composer require vlucas/phpdotenv
```

Dit installeert `phpdotenv` én voegt het toe aan `composer.json`. De `vendor/`-map wordt aangemaakt.

.gitignore aanmaken

```
vendor/  
.env
```

****Waarom `vendor/`?** Honderden bestanden die jij niet geschreven hebt. Iemand die jouw project wil draaien voert `composer install` uit en krijgt ze automatisch terug.

****Waarom `.env`?** Hier komen je wachtwoorden in. Die horen nooit in git. Elke ontwikkelaar en elke server heeft zijn eigen `.env`.

.env aanmaken

```
DB_HOST=localhost  
DB_USER=root  
DB_PASS=  
DB_NAME=user_repo_opdracht
```

index.php aanpassen

```
<?php  
  
require_once __DIR__ . '/vendor/autoload.php';  
  
$dotenv = Dotenv\Dotenv::createImmutable(__DIR__);  
$dotenv->load();  
  
$pdo = new PDO(  
    'mysql:host=' . $_ENV['DB_HOST'] . ';dbname=' . $_ENV['DB_NAME'],  
    $_ENV['DB_USER'],  
    $_ENV['DB_PASS']  
);  
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);  
  
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');  
$stmt->execute(['peter']);  
  
var_dump($stmt->fetch());
```

`__DIR__` geeft altijd het absolute pad van de map waar dit bestand staat. Zonder `__DIR__` werkt het pad alleen als de browser het bestand precies vanuit die map aanroept — met `__DIR__` werkt het altijd.

Ververs de browser. Het werkt nog steeds, maar je wachtwoord staat nu veilig buiten je code.

Stap 3: Alles op één plek

Probleem: straks heb je meerdere PHP-pagina's. Al die pagina's hebben de autoloader en `.env` nodig. Nu moet je die twee regels op elke pagina herhalen.

Oplossing: één `bootstrap.php` die je overal inlaadt. Alle initialisatie op één plek — als er iets verandert, pas je het op één plek aan.

bootstrap.php aanmaken

```
<?php

require_once __DIR__ . '/vendor/autoload.php';

$dotenv = Dotenv\Dotenv::createImmutable(__DIR__);
$dotenv->load();
```

index.php vereenvoudigen

```
<?php

require_once __DIR__ . '/bootstrap.php';

$pdo = new PDO(
    'mysql:host=' . $_ENV['DB_HOST'] . ';dbname=' . $_ENV['DB_NAME'],
    $_ENV['DB_USER'],
    $_ENV['DB_PASS']
);
$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute(['peter']);

var_dump($stmt->fetch());
```

Ververs de browser. Werkt nog steeds.

Stap 4: De Database-klasse

Probleem: de PDO-initialisatie staat in `index.php`. Straks hebben meerdere pagina's een verbinding nodig — dan schrijf je dat blok code steeds opnieuw.

Oplossing: een `Database`-klasse die de verbinding één keer aanmaakt en daarna hergebruikt.

Maak de map `src/Config/` aan en daarin `Database.php`:

```

<?php

namespace App\Config;

use PDO;

class Database
{
    public static PDO $pdo;

    public static function getInstance(): PDO
    {
        if (!isset(self::$pdo)) {
            self::$pdo = new PDO(
                'mysql:host=' . $_ENV['DB_HOST'] . ';dbname=' . $_ENV['DB_NAME'],
                $_ENV['DB_USER'],
                $_ENV['DB_PASS']
            );
            self::$pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
        }

        return self::$pdo;
    }
}

```

Er staan twee nieuwe dingen bovenaan: `namespace` en `use`. Die verdienen uitleg.

Namespaces

`namespace App\Config;` is het adres van deze klasse. Het voorkomt naambotsingen en vertelt de autoloader waar het bestand staat. De namespace volgt altijd de mappenstructuur binnen `src/`:

Namespace	Bestand
<code>App\\Config\\Database</code>	<code>src/Config/Database.php</code>
<code>App\\Domain\\User</code>	<code>src/Domain/User.php</code>
<code>App\\Repositories\\UserRepository</code>	<code>src/Repositories/UserRepository.php</code>

Dankzij de PSR-4 instelling in `composer.json` vindt PHP deze bestanden automatisch — geen `require_once` nodig voor je eigen klassen.

Waarom use PDO?

Zodra je een namespace declareert, zoekt PHP alle klassenamen relatief aan die namespace. Schrijf je `new PDO(...)` in `namespace App\Config`, dan zoekt PHP naar `App\Config\PDO`. Dat bestaat niet.

PDO is een ingebouwde PHP-klasse in de **globale** namespace. Door `use PDO` bovenaan te zetten vertel je PHP: als ik `PDO` schrijf, bedoel ik de globale `PDO`.

Dit geldt voor alle ingebouwde PHP-klassen in een namespace: `PDO`, `PDOException`, `DateTime`, enzovoort.

static, self:: en \$this

Normaal maak je een object aan met `new`. Bij `Database` doe je dat nooit:

```
$pdo = Database::getInstance(); // geen new, geen object
```

Waarom? Je wil maar één databaseverbinding in je hele applicatie. Elke keer dat je `getInstance()` aanroept, krijg je dezelfde verbinding terug — het **singleton-patroon**. De check `!isset(self::$pdo)` zorgt dat de verbinding maar één keer wordt aangemaakt.

Een statische property of methode hoort bij de **klasse zelf**, niet bij een specifiek object:

Schrijfwijze	Betekenis
<code>\$this->naam</code>	dit specifieke object — alleen beschikbaar na <code>new ...</code>
<code>self::\$naam</code>	de klasse zelf — altijd beschikbaar, ook zonder object

Je kunt `$this` niet gebruiken in een `static` methode — er is geen object aangemaakt met `new`.

Property vs lokale variabele

```
class Database
{
    public static PDO $pdo; // PROPERTY: hoort bij de klasse, altijd beschikbaar

    public static function getInstance(): PDO
    {
        $dsn = 'mysql:host=...'; // LOKALE VARIABELE: bestaat alleen in deze methode

        if (!isset(self::$pdo)) {
            self::$pdo = new PDO($dsn, ...);
        }

        return self::$pdo;
    }
}
```

`$dsn` verdwijnt zodra de methode klaar is. `self::$pdo` blijft bestaan zolang het script draait. Hetzelfde geldt met `$this` in gewone klassen:

```

class Voorbeeld
{
    private string $naam; // PROPERTY: beschikbaar via $this->naam overal

    public function __construct(string $naam)
    {
        $this->naam = $naam; // $this->naam = property | $naam = lokale parameter
    }

    public function begroet(): string
    {
        $groet = 'Hallo, '; // LOKALE VARIABELE: alleen hier
        return $groet . $this->naam;
    }
}

```

index.php aanpassen

```

<?php

require_once __DIR__ . '/bootstrap.php';

use App\Config\Database;

$pdo = Database::getInstance();

$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute(['peter']);

var_dump($stmt->fetch());

```

Ververs de browser.

Stap 5: SQL uit index.php

Probleem: de query staat nog in index.php. Straks heb je tien queries door elkaar staan.

Oplossing: een User-klasse en een UserRepository.

User

Maak de map src/Domain/ aan en daarin User.php:

```

<?php

namespace App\Domain;

class User
{
    public function __construct(
        public readonly int $id,
        public readonly string $username,
        public readonly string $email
    ) {}
}

```

UserRepository

Maak de map `src/Repositories/` aan en daarin `UserRepository.php`:

```

<?php

namespace App\Repositories;

use App\Domain\User;
use PDO;

class UserRepository
{
    public function __construct(private PDO $pdo) {}

    public function findByUsername(string $username): ?User
    {
        $stmt = $this->pdo->prepare('SELECT * FROM users WHERE username = ?');
        $stmt->execute([$username]);
        $row = $stmt->fetch();

        if ($row === false) {
            return null;
        }

        return $this->hydrate($row);
    }

    private function hydrate(array $row): User
    {
        return new User(
            id: (int) $row['id'],
            username: $row['username'],
            email: $row['email']
        );
    }
}

```


`use App\Domain\User` importeert je eigen klasse. `use PDO` importeert de ingebouwde PDO-klasse — hetzelfde principe als in de `Database`-klasse.

index.php — eindresultaat

```
<?php

require_once __DIR__ . '/bootstrap.php';

use App\Config\Database;
use App\Repositories\UserRepository;

$pdo = Database::getInstance();
$repository = new UserRepository($pdo);

var_dump($repository->findByUsername('peter'));
```

Stap 6: Terugkijken

Zo begon je in stap 1:

```
$pdo = new PDO('mysql:host=localhost;dbname=user_repo_opdracht', 'root', '');
$stmt = $pdo->prepare('SELECT * FROM users WHERE username = ?');
$stmt->execute(['peter']);
var_dump($stmt->fetch());
```

Zo ziet `index.php` er nu uit:

```
require_once __DIR__ . '/bootstrap.php';

use App\Config\Database;
use App\Repositories\UserRepository;

$pdo = Database::getInstance();
$repository = new UserRepository($pdo);

var_dump($repository->findByUsername('peter'));
```

`index.php` weet niets meer van SQL, wachtwoorden of hoe de verbinding werkt. Elke verantwoordelijkheid heeft een eigen plek:

```
Environment/
├─ src/
│ ├─ Config/
│ │ └─ Database.php ← verbinding beheren
│ └─ Domain/
│   └─ User.php ← data representeren
└─ Repositories/
    └─ UserRepository.php ← SQL schrijven
├─ vendor/
├─ .env ← geheime configuratie
├─ .gitignore
├─ bootstrap.php ← initialisatie
├─ composer.json
└─ index.php ← entry point
```

Uitbreidingsopdrachten

- 1 Voeg `findById(int $id): ?User` toe aan de repository.
- 2 Voeg `findAll(): array` toe — geeft alle gebruikers terug als een array van User-objecten.
- 3 Maak een tweede pagina `users.php` die alle gebruikers uitprint met ID en username. Het bestand mag alleen `require_once`, `use`-statements en aanroepen van de repository bevatten.